

Calibrated Scoping of Application-Layer DDoS Mitigation with a Session-Centric Knowledge Graph

Natanael Oliveira, Arthur Kobielski, Marcos Luna, Luigi Carro, Jéferson Campos Nobre, and Eder John Scheid
Institute of Informatics, Universidade Federal do Rio Grande do Sul (UFRGS), Porto Alegre, Brazil
{natanael.oliveira, arthur.kobielski, marcos.luna, carro, jcnobre, ejscheid}@inf.ufrgs.br

Abstract—A distributed Slow HTTP DoS campaign keeps every source below per-source thresholds, so mitigation turns on scope: which clients to block, sparing legitimate users. Over a session-centric knowledge graph, a binomial enrichment test blocks a TLS fingerprint only when its share of an alarm’s origins is improbable under normal traffic, at a level calibrated to a false-alarm budget. On generated traffic the natural scope, the cluster’s most common fingerprint, blocks 0% of a botnet on five or more stacks and 39% of legitimate traffic. The test blocks 90% up to 25 stacks with no observed collateral, as does an unseen-fingerprint filter at 0.03%, and 85% at 2% collateral on stacks real clients share, where that filter blocks none. A learned cross-session model (AUC 0.98) falls to 0.48–0.75 on another stack count. On eight days of four CDN endpoints, synchronized client fleets set a calibration floor: the smallest botnet the binomial names is 16% of the busiest endpoint’s window and 4–12 windows on the small ones. A beta-binomial modeling the fleets lowers it to 3% and 0.9–2.8 windows, at the operating point of a z -score calibrated to the same budget, 0.4% of clean windows. Every calibrated scope stops 67–71% of a botnet as large as the busiest endpoint’s window. A fresh day analyzed as planned in advance gives the binomial 2 false alarms in 1 152 windows. The ontology compiles to a count query reproducing the operator’s origin and /24-pair counts on two days, and the scope exports as STIX 2.1.

Index Terms—security management, knowledge graph, application-layer DDoS, semantic models, automated reasoning, explainability, mitigation policy

I. INTRODUCTION

Application-layer Distributed Denial of Service (DDoS) attacks continue to grow in volume and frequency. In August 2023, HTTP/2 Rapid Reset (CVE-2023-44487) pushed mitigation peaks to 398 million requests per second [1], [2]. A peak of 201 million came from about twenty thousand machines [3], and Cloudflare blocked more than 6 500 hyper-volumetric attacks in Q2 2025 [4]. Volumetric defenses absorb attacks of this magnitude. Detection is harder at the low-rate end of the spectrum, where *distributed Slow HTTP DoS* sits. Slowloris and its descendants, plus rate-based variants that still hold connections open, such as HULK and GoldenEye [5], exhaust a server from several coordinated origins, each sending requests small enough that no per-source threshold is crossed.

A stealthy campaign also shapes each session to look legitimate on the per-session signals detectors rely on, such as connection duration and route entropy [6]. What remains discriminative lives in the structure *between* sessions: the same

client fingerprint recurring across many origins, and those origins converging on one endpoint. Framed this way, the problem belongs to network and service management as much as to detection, because acting on it requires a decision: *which* clients to treat, on *what* evidence, and with a filter narrow enough to spare the legitimate users of the attacked service.

Existing work addresses the detection side of this problem and leaves the operational side open. A meta-analysis of 75 studies finds that 47% of application-layer DDoS detectors derive their features from HTTP sessions [7], flattened into a numeric vector before the classifier, which then cannot reason about fingerprints recurring across origins or about patterns that exist only between individually normal sessions. Graph-based detectors keep such relations: KLAG [8] classifies the nodes of knowledge graphs built from network logs and reports in natural language. Three opportunities remain open. First, the reasoning unit is the *network node* (port, IP, flow), while Layer-7 coordination becomes visible only at the level of the application session. Second, coordination is captured implicitly, in learned embeddings, so no operator can inspect the relations behind the verdict. Third, the output stops at a report, whereas acting on it demands a mitigation policy whose scope follows from the graph.

This paper addresses them by treating the scope of the mitigation as a calibrated statistical test, run over a knowledge graph of HTTP sessions. Each HTTP/TLS request resolves to an `ApplicationSession` node of an OWL graph kept over a sliding window, and the sessions are linked by sub-properties weighted by the attacker’s cost of breaking each signal. A SPARQL/SWRL rule raises the alarm, whose scope is the set of fingerprints a binomial enrichment test finds over-represented against the endpoint’s normal traffic, at a level calibrated to a false-alarm budget. We target campaigns run from botnets of compromised devices, whose TLS stacks their operator cannot cheaply replace, and evaluate bots that present a browser’s fingerprint as the boundary of the method. This paper makes four contributions. (i) We show that the natural scope, the property most of the alarm’s cluster shares, blocks users and no attackers once a botnet spans a few TLS stacks. Scoping by enrichment against an attack-free profile blocks the attack, stacks that real clients share included, which a filter of unseen fingerprints misses, and it needs no labels, whereas a learned cross-session model fails on a botnet with

TABLE I
COMPARISON OF RELATED WORK

Work	Year	Objective	Reasoning Unit	Cross-session Relation	Verdict Explanation	Mitigation Scope	Collateral
[9]	2012	L7 DDoS	Session as Vector	-	-	-	-
[6]	2023	L7 DoS	Session as Vector	-	-	-	-
[10]	2024	Bot Traffic	JA4 Fingerprint	Proprietary	-	Undisclosed	Undisclosed
[8]	2026	Network Threats	Network Node	Implicit (Learned)	Post-hoc (LIME/LLM)	-	-
This Work	2026	Distributed L7 DDoS	Application Session	Explicit (Typed and Weighted)	Derivation	Derived	Reported

another number of stacks. (ii) On eight days of a CDN’s traffic we measure what calibrated scoping can reach. Legitimate fleets set a floor on the smallest botnet the test names, a beta-binomial background that models them lowers it several times, and in the stealthy regime the scope names the botnet that the volume trigger mostly misses. We compare the test with baselines calibrated to the same budget and check its false-alarm rate on a fresh day analyzed with the configuration fixed in advance. (iii) We specify the session graph as an OWL ontology with *six typed sub-properties*, three of them exercised, from which the count query is compiled. On two production days it reproduces the operator’s exports window for window, and the scope exports with its evidence chain as STIX 2.1. (iv) We report the false-positive rate of the resulting filter as collateral damage on legitimate traffic, in clean and in attack windows.

Section II reviews related work, Section III presents the framework and Section IV the evaluation methodology. Section V reports the results, Section VI discusses operational placement and limitations, and Section VII concludes.

II. RELATED WORK

Table I compares this work with prior art on four dimensions: the reasoning unit, the representation of relations between sessions, the form of the explanation, and whether a mitigation scope and its cost follow from it.

A. Knowledge Graphs for Security and Management

The first wave of security knowledge graphs was *static*: entities extracted from Common Vulnerabilities and Exposures (CVE), reports and blogs, normalized and linked into a queryable base [11], [12]. Defensive knowledge follows the same pattern in D3FEND, an OWL graph of countermeasure techniques and the artifacts they act on, mapped to ATT&CK [13]. Graphs built from threat-intelligence text do not capture runtime traffic structure, and how to apply them in real attack and defense operations remains unclear [14].

More recent work populates graphs from the traffic itself, and KLAG [8] is the closest of these to the approach proposed here. Learned models are also gaining ground in adjacent management tasks: one line identifies device dependencies through link prediction over network graphs [15], another learns attack action sequences from testbed measurements with statistical models [16], and graph-based bot detection classifies the hosts of a flow graph by degree, clustering and centrality features [17]. All three operate on network-level entities, and none derives a mitigation scope from what it

finds. Account-level detectors likewise find coordination by clustering accounts that act in lockstep [18].

B. Layer-7 HTTP DDoS Detection

The state of the art on Layer-7 HTTP DDoS detection [5], [19] falls into two categories: (i) statistical profiling and (ii) machine learning (ML). The first applies Principal Component Analysis (PCA) either to flow statistics [20] or to a per-session behavioral matrix [9], the latter being among the first formulations to put the session at the center, though still as a feature vector. The second and more recent category applies supervised learning over session features [6], whose authors note that the method was not validated in real time and that its output is binary, with no ontological explanation. Neither category retains a relation between sessions: whatever coordination exists is dissolved into the feature vector before the classifier ever sees it. Defenses against attacks that mimic flash crowds admit clients that solve a puzzle or spend bandwidth, and they measure the service legitimate users keep [21], [22]. Deriving a filter from an alarm is the goal of anomaly extraction, which isolates the feature values over-represented in anomalous traffic [23], and of signature generation checked against a pool of normal traffic [24]. Both operate on flow or payload features, without the fingerprints of an application’s clients.

C. Session Modeling in Anomaly Detection

None of the 75 studies of a meta-analysis on the topic keeps the session as an object with identity, target, behavior and typed relations [7].

Industrial bot-management platforms do exploit cross-session structure: signals aggregated per JA4 over recent traffic feed bot scoring and DDoS protection [10]. They are proprietary, with no public ontology, no reproducible academic benchmark and no formalized collateral-damage metric. The channel a scope would travel on is standardized: the IETF DOTS signal channel carries a mitigation request from the attacked domain to a mitigator, scoped by the resources under attack. Its data channel installs filters on network and transport header fields, and its telemetry reports top talkers by source prefix [25]–[28]. No DOTS attribute can express an application-layer client fingerprint such as JA4.

Table I makes the boundary explicit. Statistical and machine-learning detectors reduce the session to a vector, so no relation between sessions survives and no mitigation scope or cost can follow from the decision. Graph-based detectors keep relations, on network-level entities and in learned

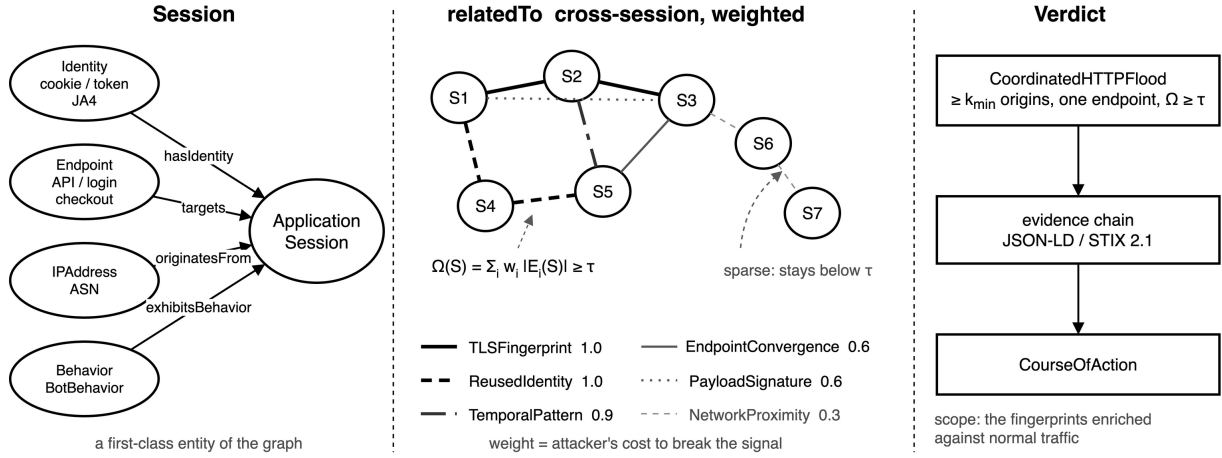


Fig. 1. Session-centric ontology and detection rule. Endpoint convergence links every pair of sessions on the endpoint. A few are drawn.

form, and industrial platforms act on per-fingerprint aggregates without disclosing the representation or the cost of acting on it. No published approach, therefore, holds three properties together: the application session as the reasoning unit, explicit and typed relations between sessions, and a mitigation scope derived from the representation that produced the verdict, with its collateral damage reported. The rest of this paper supplies that combination and measures what a wrong scope costs.

III. SESSION-CENTRIC KNOWLEDGE GRAPH

Fig. 1 summarizes the framework. Four stages run over live traffic: HTTP events are ingested, lifted into ontology instances with sessions as persistent graph nodes, evaluated by rules, and turned into a verdict with its evidence chain and scope.

A. Employing Knowledge Graphs

A knowledge graph states facts as RDF triples (subject, predicate, object), and an OWL 2 ontology declares the classes and properties they may use. We work in the OWL 2 RL profile [29], whose polynomial forward-chaining entailment makes bounded materialization tractable for runtime reasoning where OWL 2 DL is not, and materialization runs once per window, off the per-request path. SWRL adds Horn rules, which assert a triple when a conjunction of premises holds, and SPARQL queries and aggregates the result. Client identity comes from JA4 [30], a hash of the TLS *ClientHello* that identifies a client’s TLS stack. Being independent of the source address, it survives IP rotation.

B. Defining the Ontology

The central class is *ApplicationSession*, related by *hasIdentity* to a composite *Identity* (cookie, JWT token, username, TLS fingerprint), by *originatesFrom* to its source *IPAddress*, by *targets* to an *Endpoint* (*LoginEndpoint*, *APIEndpoint*, *StaticEndpoint*), by *exhibitsBehavior* to a behavioral profile, and by *mitigatedBy* to a *Mitigation* (*RateLimitPolicy*, *ChallengeResponse*, *WAFRule*). The evidence chain carries its scope (Section III-G). The abstract class *ApplicationLayerAttack*

specializes into *SlowHTTPDoSFamily*, itself split into *ConnectionExhaustionAttack* (*Slowloris*, *slow body*, *slow read*) and *CoordinatedHTTPFlood* (*HULK*, *GoldenEye*), the class our rule derives. The ontology is aligned with STIX 2.1 [31] and MITRE ATT&CK.

C. Threat Model

The defender operates the attacked application and observes, per request, the TLS *ClientHello* (hence a JA4 fingerprint), source address and prefix/ASN, target endpoint, arrival time, and whatever identity material the application exposes. It sees neither the botnet’s control channel nor which sources are coordinated, but can profile the service’s own traffic outside attack episodes. The attacker controls K compromised devices and may shape each session to be statistically indistinguishable from a legitimate one, the stealth assumption of our evaluation. The damage is exhausted capacity: each session holds a connection and a worker as a legitimate one does, so no per-session rate rises, and the aggregate grows only by the campaign’s share of the endpoint’s traffic. Timing, identities and payloads can be jittered, rotated or randomized, each at an operational cost that the weighting of Section III-D encodes. The TLS stack is harder to change, because it is fixed by the library each device ships, as in the router, camera and DVR botnets of the Mirai lineage [32]. Presenting a browser’s *ClientHello* would require shipping an impersonation library to every device and keeping it in step with browser releases. Bots that do so, such as headless-browser farms or bots built on TLS-impersonation libraries, fall outside this model, and Sections V-C and VI evaluate them as its boundary.

D. The Weighted *relatedTo* Family

Cross-session relatedness is a structured family of relations, declared via *rdfs:subPropertyOf* under a symmetric *relatedTo* and annotated with a *coordinationWeight* $w_i \in [0, 1]$ (Fig. 1). Each is instantiated independently from its own observed evidence, so two sessions may share one, several or all six. Three of the six are exercised here (Appendix C).

The weights follow the attacker’s cost of breaking each signal. A TLS stack is fixed by each device’s library, and JA4 sorts extensions, so it survives the extension randomization that breaks JA3 [30]. Botnets spread across prefixes, and carrier-grade NAT makes a shared /24 a poor discriminator, which leaves network proximity the weakest link. Appendix B corroborates the ordering, leaving the absolute values open.

E. Detection Rule

Let S_W be the sessions active in the window W . For $S \subseteq S_W$ and a candidate endpoint e , define the *coordination mass*

$$\Omega(S) = \sum_{i \in \mathcal{R}_{\text{rel}}} w_i |E_i(S)|, \quad (1)$$

with $E_i(S)$ the set of pairs of distinct origins $\{o(s_a), o(s_b)\}$, $s_a, s_b \in S$, whose sessions are linked by sub-relation i , where $o(s)$ is the source address of s . Each pair counts once, so one client’s many sessions cannot pose as coordination. The rule CoordinatedHTTFFlood holds when there exists S with (i) at least $k_{\min}$ origins; (ii) $\langle s, \text{targets}, e \rangle$ for every $s \in S$; and (iii) $\Omega(S) \geq \tau_{\text{cluster}}$. It mitigates only when the enrichment test of Section III-G names a fingerprint. Per window, S is every session to e (Appendix E).

At window scale this mass is volume: under condition (ii) the endpoint term, $0.6 \binom{n}{2}$ for n origins, carries 91% of Ω in the campaign of Listing 2 (Appendix D). A distinct-origin threshold calibrated like τ_{cluster} therefore gates at least as well (Appendix E), and the test supplies the discriminator.

F. Runtime Graph Construction

Sessions and relations live inside a sliding operational window ($W = 5$ min by default) and are purged incrementally, following an RDF stream-processing discipline, with materialization over Apache Jena Fuseki and TDB2 storage in the prototype. The evaluation computes the same rule from class counts (Appendices D and E). Appendix A gives the construction algorithm and the per-pair procedure of each `relatedBy` sub-relation. Exact JA4, endpoint and prefix matches are equivalence relations, so each partitions S into classes and $|E_i(S)| = \sum_k \binom{n_k}{2}$ over their sizes n_k in origins. Admitting a session then adds its origin to one class per such sub-relation, a hash-set insertion independent of $|S_W|$, and $\Omega(S)$ needs no pair edge. Only the non-transitive relations (near-variant JA4, temporal, payload) compare candidate pairs, which prefix indexes and locality-sensitive hashing restrict. Section V-E measures both paths against window occupancy. SWRL states each `relatedBy` sub-relation as a Horn rule over the predicates the evidence extractors produce, numeric ones such as the dynamic time warping (DTW) distance entering as a built-in comparison against a threshold. Since SWRL cannot sum, a SPARQL query aggregates the class sizes, weighted by the `coordinationWeight` annotations (Listing 1).

G. Evidence Chain and Derived Mitigation Scope

When the rule fires, the engine emits the satisfied rule, the instances involved, the decomposition of $\Omega(S)$ per sub-relation, and the derived mitigation scope. We export it as

JSON-LD and as STIX 2.1, an *indicator* plus a *course-of-action* linked by *mitigates*, for SIEM and SOAR ingestion.

How the scope is chosen decides whether the verdict is useful or harmful, and the frequency-based choice fails. The modal fingerprint of the cluster selects whatever is common, and on a service under attack what is common is the legitimate population. Against a botnet spread over several TLS stacks, each attacker stack is smaller than the head of the benign distribution, so the modal value is a *legitimate* fingerprint and the resulting filter blocks users and no attackers (Section V-C).

We therefore select the discriminator by **enrichment**. Let $c(f)$ be the number of origins of the fired cluster that present fingerprint f , $n = \sum_f c(f)$, and $b(f)$ the prevalence of f , plus $1/N$, among the N origin–fingerprint pairs of a profile of normal traffic kept by the operator outside attack episodes. The scope admits every f that is enriched, $c(f)/n \geq \rho b(f)$, and improbable under the background, $\Pr[X \geq c(f)] < \lambda_e/|F|$ for $X \sim \text{Bin}(n, b(f))$, where the Bonferroni divisor $|F|$ counts the fingerprints of profile and cluster. The result, a *set* of fingerprints, covers a fragmented botnet. Fleets of legitimate clients that switch on together break the binomial’s independence on production traffic (Appendix E). The level λ_e is therefore calibrated per endpoint e as τ_{cluster} is: the largest value up to 0.01 at which the scope names a filter in at most 1% of attack-free windows. This gives the test a floor: among n origins it names a stack only past the smallest count c with $\Pr[X \geq c] < \lambda_e/|F|$, so the smallest nameable botnet grows as the level falls (Section V-D). Because fleets also make counts overdispersed, we evaluate a variant whose background is a beta-binomial fitted per fingerprint on the calibration windows. We use $\rho = 3$. On generated traffic, one origin per session, λ_e stays at 0.01.

The background must come from outside the attack episode, because the campaign spans the whole window and would otherwise give every candidate an enrichment of one. No labels are involved: the profile is what a defender sees in normal operation. An adversary hiding inside a popular benign fingerprint is therefore not enriched, so the framework refuses the scope and reports that no discriminator exists.

IV. EVALUATION METHODOLOGY

Each traffic source answers one question. A generator calibrated on benign CICIDS2017 sessions and on JA4 measured at a production CDN carries the controlled campaigns of the main evaluation. Laboratory captures test conventional attacks and the closest published system, and eight days of that CDN’s traffic test the rule’s false alarms and, with a botnet injected, its detection. We then present the baselines, the ablation configurations and the metrics.

A. Scenarios and Datasets

The evaluation is parameterized by the *degree of distribution* K of the campaign, the number of devices the attacker controls. We report $K = 50$, a small botnet whose origins each stay inside their own rate limit, and $K = 1000$, spread over hundreds of ASNs and prefixes, with negligible per-session

signal and a nearly empty `relatedByNetworkProximity`. A single source is left to volumetric detectors.

Primary evaluation: calibrated synthetic traffic. A generator produces distributed Slow HTTP DoS campaigns, the family of Section III-B: sustained low-rate sessions against one endpoint, with known coordination ground truth, parameterized by K , TLS and identity sharing, IP/ASN dispersion and the time window.

Two parameters of that generator decide whether the scenario resembles production: benign fingerprint popularity, which we measure, and botnet composition, which no published measurement fixes and we therefore sweep. The first is shaped as a Zipf curve of exponent α : real populations are concentrated, since an up-to-date browser on a given platform emits essentially one JA4 [30]. We calibrated that curve against 6.33M TLS requests logged at a production edge point of presence of Azion Technologies¹: 495 distinct fingerprints, the most common accounting for 38.4% of requests and the top ten for 93.8%. The measured curve is bracketed by $\alpha = 1.5$ and $\alpha = 2.0$. We take $\alpha = 1.5$ as canonical and sweep the range. As for botnet composition, real botnets span device classes such as routers, IP cameras and DVRs [32], and therefore TLS stacks. We distribute 90% of the campaign uniformly over M stacks, the conservative choice for a given M , and the rest over one-off fingerprints, and take $M = 25$ as canonical. An *adversarial* mode lets the botnet adopt the most common benign fingerprints, as an impersonating bot would, and marks the boundary of the threat model. Appendix B reports the measurement and its caveats.

Legitimate distributions are calibrated against the $\sim 322k$ benign sessions of CICIDS2017 and verified by Kolmogorov–Smirnov tests: $D = 0.003$ for duration and $D = 0.002$ for request count, both with $p > 0.8$, so their marginals match CICIDS2017’s. A *stealth* mode draws attacker sessions from the same distributions as benign traffic, so the campaign reveals itself only through cross-session structure.

Laboratory captures. CICIDS2017 [33] (Wednesday capture: Slowloris, Slowhttptest, HULK, GoldenEye) and CIC-IoT2023 [34] (DDoS Slowloris and HTTP Flood), the latter being the dataset on which KLAGe reports $F_1 = 84.1\%$ for DDoS Slowloris, run through the same extraction pipeline (PCAP $\rightarrow$ JA4 + flows $\rightarrow$ sessions $\rightarrow$ graph). **Production traffic.** Eight days of Azion’s CDN traffic, as counts of distinct origins, connections and same-/24 pairs per fingerprint and five-minute window, and a ninth analyzed with every choice fixed in advance (Section V-D).

B. Baselines, Configurations and Metrics

Each ablation configuration is run with $n = 30$ distinct seeds, each campaign split 70/30 into training and test sessions, giving bootstrap intervals and paired tests. All configurations consume the same traffic attributes, so the ablation isolates *representation*. τ_{cluster} is fixed per scenario at the 99th

¹Measured 22 August 2026 from an access log the platform already collects. The extract holds only client-software fingerprints and their frequencies, with no address, host, URI, header or locale.

percentile of Ω over legitimate clusters, and λ_e so that the scope names a filter in at most 1% of them.

The per-session baseline is deliberately **strong**: a Random Forest (RF) over all eight to nine available flow attributes (request count and duration, byte and packet sums, inter-request time). To keep the negative result from depending on one hypothesis class, we repeat it with histogram gradient boosting, a multilayer perceptron and logistic regression over the same features. We also reimplement three academic baselines: PCA profiling [20], PCA followed by k -means clustering over a behavioral matrix [9], and supervised RF/SVM over session features [6]. The four ablation configurations are: (a) ML baseline without ontology; (b) ontology with *no* `relatedBy` sub-relation, sessions as isolated nodes; (c) ontology with *only* `relatedByNetworkProximity`, emulating a naive ASN/prefix detector; and (d) the full cross-session representation over the three sub-relations observable in the evaluated traffic.

The scope is compared with simpler ones on the same alarm: the modal fingerprint of the cluster; a filter of fingerprints absent from the profile and seen in at least $k_{\min}$ origins (*unseen*); and a per-fingerprint z -score against the profile ($z > 3$). On production traffic, that z -score and one of each fingerprint against its own calibration history also run calibrated, their thresholds raised per endpoint to the enrichment test’s budget, a filter in at most 1% of attack-free windows. Generated botnet stacks never occur in the benign vocabulary. A *shared* mode draws them from the profile past its ten most common fingerprints, as the production injection does.

Configuration (d) consumes the cross-session evidence as *features*: the share of a cluster sharing one JA4 or /24, plus its size. This feature-level proxy for the `relatedBy` family keeps every configuration equal in data and classifier.

Beyond standard classification metrics we report **collateral damage**: the fraction of legitimate traffic falling inside the scope derived from the verdict. It is the false-positive rate of the filter an operator would deploy, which can differ sharply from that of the detector (Section V-C).

V. RESULTS

The results follow the claims they support: the ablation under stealthy distributed campaigns, the rule and its scope on generated traffic, what the scope costs in collateral, the scope on production traffic, and the runtime cost.

A. Ablation Under Stealthy Distributed Campaigns

Per-session detection sits at chance over stealthy distributed campaigns whose legitimate users access the *same* endpoint and port as the attack, so that neither flow volume nor destination port separates the classes. At $K = 50$ and $K = 1000$, configuration (a) with the full flow-feature set reaches a per-session ROC AUC of 0.498 and 0.503, (b) 0.500 and 0.502, and the three academic baselines 0.495–0.518, and (a) stays at 0.488–0.503 under every classifier family ($\alpha = 1.5$, $M = 25$, $n = 30$ seeds). Each attacker session is drawn from the benign distributions by construction,

TABLE II
THE ENRICHMENT SCOPE OF THE LARGEST- Ω CLUSTER AS A DETECTOR.

Scenario	Rule, no labels			Learned, rec. @ FPR= 0		
	Recall	FPR	F_1	Unseen	(d)	+ profile
Monolithic ($M=1$)	89.8%	0.00%	0.946	89.8%	91.7%	85.9%
$M=5$	90.0%	0.00%	0.948	90.0%	88.6%	91.7%
$M=25$	90.3%	0.00%	0.949	90.3%	36.4%	87.4%
$M=100$	38.6%	0.00%	0.556	88.1%	17.6%	86.9%
$M=25$, shared	85.4%	2.23%	0.910	0.0%	—	—
$M=25$, adversarial	30.4%	3.78%	0.452	0.0%	7.8%	6.6%

Unseen: recall of the filter of fingerprints absent from the profile (FPR at most 0.03%). Shared: stacks drawn from the profile's tail (drawn instead from the generator's vocabulary, mostly absent from the sampled profile, the unseen filter blocks 84.4% and the rule 89.8%); not run for the learned models, which train on the campaign's labels, the second also reading the rule's profile. $K = 1000$, $n = 15$ seeds.

so the collapse lies in the representation. The cross-session representation (d) reaches 0.927 and 0.982, and (c), restricted to network proximity, only 0.499 and 0.659, because the botnet is dispersed. Paired Wilcoxon tests, Bonferroni-corrected over four comparisons, make (d)–(c) and (d)–(a) significant at $K = 1000$ ($p_{\text{Bonf}} = 7.5 \times 10^{-9}$, Cohen's $d = 13.5$ and 22.3), a difference stable across draws of one generator.

That separability does not transfer. Fragmenting the botnet over M stacks makes the cross-session evidence *non-monotonic* in the label: at $K = 1000$ an attacker shares a fingerprint with a few dozen peers, a legitimate client in the benign head with hundreds. Tree ensembles carve out that band (0.92–0.99), the perceptron reaches 0.86 and 0.95, and logistic regression, whose response to each attribute is monotone, 0.87 and 0.80. Trained on campaigns of five stacks and tested on 25 or 100, (d) falls to 0.61 and 0.63, and trained on 25 and tested on five, to 0.48 (Appendix C): the forest learns the band one M produces. The rule needs no labels: it reads a profile of normal traffic and keeps about 90% at every M up to 25 (Table II).

That (b) sits at chance too shows per-session attributes are not enough: cross-session evidence carries detection. The seven attack clusters of the two laboratory captures, six attacks and the DoS-Other class of CICIDS2017, come from one to seven origins in one /24, floods within reach of a per-prefix limit. On those captures a strong per-session classifier suffices ($F_1 = 0.900$ on CIC-IoT2023 DDoS Slowloris, Appendix C). The evidence chain of Listing 2 is therefore that of a canonical generated campaign.

B. The Rule Itself as a Detector

Every number so far came from a classifier. Here the sessions matching the scope of Section III-G are the flagged set, with no training or label, and the classifier is forced onto the rule's operating point (Table II).

Against a monolithic or lightly fragmented botnet the learned model is competitive at zero false positives (91.7% and 88.6% recall against 89.8% and 90.0%). At 25 stacks the rule pulls ahead, 90.3% against 36.4%, and 66.8% for a model allowed 1% FPR (AUC 0.979). Given the rule's profile as two per-session features, the fingerprint's prevalence and its enrichment in the cluster, the model recovers 87.4% at

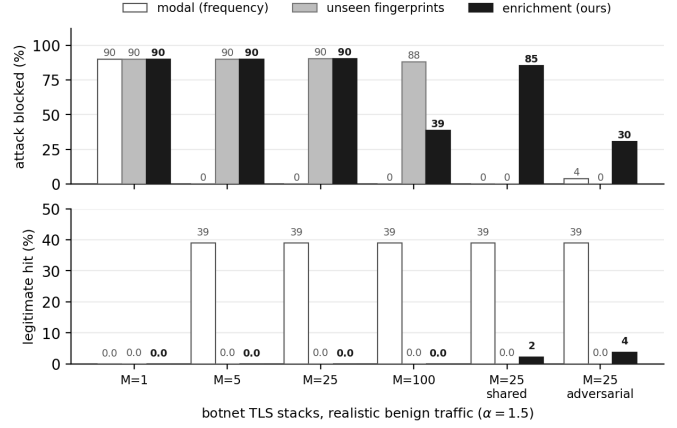


Fig. 2. Scope: frequency, unseen filter and enrichment ($\alpha = 1.5$, $n = 15$).

25 stacks and 86.9% at one hundred, where stacks too small for a thousand-session profile to name hold the rule at 38.6% (Section V-C). The model trains on labels of the campaign it scores, which no operator holds during an attack, and the rule matches it up to 25 stacks without them. In the adversarial row the rule incurs a 3.78% false-positive rate, and the model reaches 23.4% once allowed 1%. The unseen filter does as well as the rule up to 25 stacks, since generated stacks never occur in the benign vocabulary, and better at one hundred (88.1%). On stacks drawn from the profile's tail, which real clients also present, it blocks none, the rule 85.4% at 2.23% collateral, and an uncalibrated z -score 89.9% at 3.37%.

Table II takes the cluster of largest Ω as fired. Run per five-minute window (Table V), $\Omega(S) \geq \tau_{\text{cluster}}$ alone fires on 78–86% of attack windows and 0.6% of clean ones, but also on 80–100% of flash crowds [35]. Requiring the enrichment test to name a fingerprint removes every false alarm and keeps the detection, with no collateral.

C. Collateral Damage on Legitimate Traffic

A mitigation is meaningful only as a pair, what the scope blocks and what it costs (Fig. 2, $n = 15$ campaigns).

For a monolithic botnet the two agree: 89.8% of attacker sessions blocked with no collateral observed, where a global endpoint rate limit disconnects every legitimate user. Our earlier frequency rule was validated in that regime, which is not representative. From five stacks on, the modal fingerprint of the cluster becomes a *legitimate* one and the rule inverts: 0.0% of the attack blocked and 39.0% of legitimate traffic hit, a net loss. A more concentrated benign population ($\alpha = 2.0$) raises that collateral to 61.1%.

Enrichment removes the failure, blocking 90.0% and 90.3% of the attack at $M = 5$ and 25 stacks with **no collateral observed** across the $n = 15$ campaigns, and reproducing the frequency rule where it worked. The unseen filter shares this result on generated stacks and loses it on shared ones (Table II). The surviving 10% is the tail of attackers with one-off fingerprints, so scoped mitigation trades completeness for precision. At $M = 100$ coverage falls to 38.6%, still with

TABLE III
THE SCOPE ON PRODUCTION TRAFFIC, PER ENDPOINT.

Scope	Clean			New stacks		Shared	
	FA	Coll.	Flash	0.1×	1×	0.1×	1×
E1 binomial	0.0	–	0.0	11.8	70.8	7.7	68.6
beta-bin.	0.1	0.2	0.1	11.9	70.8	10.0	67.8
z-score	0.2	0.2	0.3	11.9	70.8	9.0	67.1
E2 binomial	0.3	31.4	5.4	0.0	5.0	0.0	0.0
beta-bin.	1.3	10.7	13.5	0.7	25.9	0.2	13.2
z-score	1.1	11.0	14.2	0.1	18.3	0.0	2.5
E3 binomial	0.1	48.5	11.0	0.0	1.9	0.0	0.5
beta-bin.	0.0	–	6.6	0.3	22.7	0.3	13.9
z-score	0.1	48.5	2.0	0.3	20.3	0.0	4.8
E4 binomial	0.0	–	2.2	0.1	2.1	0.0	1.5
beta-bin.	0.1	3.5	0.9	0.1	6.0	0.1	3.9
z-score	0.0	–	0.3	0.1	6.0	0.0	1.2
All binomial	0.1	32.9	4.6	3.0	19.9	1.9	17.6
beta-bin.	0.4	10.5	5.3	3.3	31.4	2.7	24.7
z-score	0.4	10.8	4.2	3.1	28.8	2.3	18.9
<i>Fresh day, all endpoints</i>							
rule	0.2	41.2	1.0	0.0	17.3	0.0	16.6
binomial	0.2	41.2	1.1	0.0	20.4	0.0	17.0
beta-bin.*	0.0	–	4.3	0.0	32.5	0.0	26.2
z-score	0.2	41.2	0.4	0.0	31.4	0.0	19.7

In %. Binomial: the enrichment test joined with the unseen filter, known fleets exempted; beta-bin.: the same over a beta-binomial background; z-score: calibrated to the same budget; all under the distinct-origin gate. Rule: the base rule under $\Omega \geq \tau$. FA: share of attack-free windows (5643 on the test days, 1152 on the fresh day) where the scope names a filter; Coll.: median share of their clients it blocks; Flash: FA with a flash crowd of 100 users. Blocked: mean share of a 25-stack botnet the size of the endpoint's median window (1×) or a tenth of it, on stacks absent from the traffic (new) or drawn from the profile past its ten most common fingerprints (shared); one-off fingerprints cap it near 90%. *Built after the fresh day was read.

no collateral: a stack of about ten sessions in two thousand cannot be told from the tail of a 1000-session profile, and pooling 30 attack-free periods into the profile restores 89.6%.

Two conditions bound the result. The first is the boundary of the threat model: when the botnet adopts common benign fingerprints, little is enriched and the rule blocks 30.4% of the attack at 3.78% collateral, against 3.6% and 39.0% for the frequency rule. With a stricter ρ the scope declines to name a fingerprint, the correct report when no discriminator exists. The second is the background profile. Moderate drift is tolerable ($\alpha = 2.0$ profile against an $\alpha = 1.5$ episode: 90.3% coverage, no collateral), while a flat or missing profile makes every fingerprint look rare and collateral jumps to 77.6%. A fresh and ample profile is thus a deployment requirement.

D. Production Traffic

Eight days of access logs from four endpoints of a CDN operator's own services test the scope on real clients (Tables III and IV). Each of the last five days is tested with the profile, thresholds and level of the days before it, with the canonical botnet injected into the counts (Appendix E). The scope names a filter in 60.1% of clean windows counted in connections, since one client opens many, and in 26.9% counted in origins at the nominal level, because legitimate fleets switch on together. Calibrating λ_e on the preceding days brings the rule to 0.2%, and a threshold on distinct origins, which fires on the same windows as Ω in 86–99.6% of cases, halves the false alarms as the gate with no loss of detection (6 against 14 windows).

TABLE IV
CALIBRATION FLOOR AND WAF AGREEMENT PER ENDPOINT.

Endpoint	Origins	λ_e		Floor (× window)			WAF
		bin.	beta	bin.	+ fleets	beta	
E1, RUM beacons	$> 10^3$	10^{-60}	10^{-4}	0.16	0.07	0.03	–
E2, web console	$< 10^2$	10^{-73}	10^{-2}	11.6	11.6	0.90	1.5%
E3, API	$< 10^2$	10^{-14}	10^{-2}	4.1	3.5	1.4	84%
E4, SSO	$< 10^2$	10^{-4}	10^{-2}	4.2	4.2	2.8	96%

Origins per window; λ_e : the calibrated level under the binomial and the beta-binomial. Floor: the smallest 25-stack botnet on new stacks whose stacks the level names, as a multiple of the median window, medians over the five test days. WAF: share of the clients the scope would block that the WAF also blocked (E1 has no WAF activity).

Fleets set a calibration floor. They push λ_e to 10^{-60} on the busiest endpoint, where the binomial test names a stack only past 15 to 18 origins, so the smallest 25-stack botnet it can name is 16% of the typical window. That floor falls to 7% once the endpoint's five or six known fleets (fingerprints the test names in at least 5% of calibration windows) are exempted from the scope and from the level's calibration, which raises λ_e to about 10^{-21} . The exemption is an allow-list: a bot presenting a fleet's fingerprint is not scoped. On the three small endpoints the floor is 4 to 12 times the whole window.

A background that models the fleets lowers the floor. Fleets make fingerprint counts overdispersed. Against a beta-binomial whose intra-window correlation is fitted per fingerprint on the calibration windows (Appendix E), the busiest endpoint's level rises to 7×10^{-5} , where three origins name a stack, and the others' to the 0.01 cap. The floor falls to 3% of the busiest window and to 0.9–2.8 windows elsewhere, and no fingerprint qualifies as a known fleet, so no allow-list is needed. On the console the test days' false alarms then exceed the budget (1.3%), as do the calibrated z-score's (1.1%).

Above the floor, a calibrated z-score matches the test. On the busiest endpoint every calibrated scope stops 67–71% of a botnet the size of its typical window, and 8–12% of one a tenth of it, on which the trigger rarely fires. On the small endpoints a botnet the size of the window lies below the binomial's floor, which stops 2–5% of it on new stacks, while the beta-binomial and a z-score calibrated to the same budget stop 6–26%. Pooled, the binomial configuration fires on 0.1% of clean windows (5 of 5643) and stops 38.4% and 77.7% of 100 and 1000 attackers on new stacks and 21.7% and 63.3% on shared ones, where the unseen filter alone stops none. The calibrated z-score and the beta-binomial fire on 0.4% (20 and 22 windows) and stop as many or more at 100 attackers (54.7% and 59.8% on new stacks, 23.7% and 40.5% on shared ones), with lighter alarms (10.8% and 10.5% of the window's clients against 32.9%). The binomial buys its lower false-alarm rate with a higher floor. In attack windows every calibrated scope blocks a median 0% of legitimate clients.

In the stealthy regime the trigger misses. A botnet a tenth of the busiest endpoint's window sits above its floor once known fleets are exempted, and the scope names it in every window, on new and on shared stacks. The distinct-

origin trigger, however, fires in 0.7–42% of those windows depending on the day, so 11.8% of the attackers on new stacks are stopped. A trigger on the scope alone, examined after the fact, stops 89.6% of them on new stacks and 61.1% on shared ones at 1.0% of the endpoint’s clean windows, and 90.0% and 78.3% at 2.1% with the beta-binomial.

A fresh day is consistent with the false-alarm rate. A ninth day, exported after every choice above and analyzed with the configuration and metrics fixed in advance (Appendix F), gives the binomial configuration 2 false alarms in 1 152 clean windows, consistent with the test days ($P = 0.27$). Both fall on the web console and block a median 41.2% of their window’s clients. That configuration stops 38.9% and 73.4% of 100 and 1 000 attackers on new stacks and 21.5% and 60.9% on shared ones, and at a tenth of the busiest window the trigger never fired. The calibrated z -score and the base rule, the protocol’s reference, fired on the same two windows, and the z -score stopped more at 100 attackers (58.3% and 26.7%). On the test days it shared 4 of the test’s 5 false alarms. The beta-binomial, built after the day was read, raised none. The compiled count query returned the day’s distinct origins and /24 pairs equal to the export in all 1 152 windows.

E. Cost and the Choice of W

Kept as classes (Appendix D), the equality-based sub-relations make admission constant at about $0.37 \mu\text{s}$ per session and the symbolic layer linear, at 26.4 s for 100 000 sessions against 52.8 s for pair edges at 1 000. The value of $\Omega(S)$ still grows with the square of the cluster, but computing it no longer does. The AUC of (d) is flat over a $30\times$ range of W , since the generated campaign forms one cluster at every W (Appendix D). The rule ran at $W = 300$ s.

VI. DISCUSSION AND LIMITATIONS

We discuss what bounds the results and what the graph adds (limitations continue in Appendix C).

The gain is regime-specific. Per-IP thresholds handle single-source floods, and a strong per-session classifier reaches $\text{AUC} \geq 0.98$ on public-dataset attacks. Only distributed and stealthy campaigns need cross-session evidence, and only botnets above the calibration floor, which the background model sets, can be scoped by fingerprint, below which the fallback is a rate limit or a challenge. Whether a campaign at the floor exhausts the service depends on its headroom, which we did not measure.

Each evaluation establishes one thing. The generator shows the mechanism under a mimicry imposed by construction, attacker sessions drawing their attributes from real benign distributions, so the non-separability of Section V-A is no artifact of a weakened baseline. Its stacks never occur in the benign vocabulary, which is why an unseen filter matches the scope there, and a model fitted to one of its stack counts does not carry to another. Production traffic measures false alarms on real clients and detection only for an injected botnet, over nine days of one operator. None covers a captured stealthy campaign.

The scope depends on an observable high-weight discriminator. As the fraction of origins sharing the TLS fingerprint falls, $\text{AUC}(d)$ decreases monotonically from 1.00 to ≈ 0.74 under randomization ($n = 10$), and the scope, which rests on JA4, loses its discriminator while the volume trigger survives. Jittered timing and rotated identities leave it intact. A bot outside the threat model that presents a browser’s JA4 defeats it. Encrypted Client Hello [36] hides JA4 from on-path observers but not from a defender that terminates TLS, as assumed here.

What the graph adds. The AUC gain of Section V-A comes from representing cross-session structure, as configuration (d) does in features (a linear model over them still attains 0.799 against 0.495). OWL adds nothing to that number or to the trigger, and the scope’s test is statistical. The graph is the specification the operator deploys: the ontology marks what each equality sub-relation equates, its weight and the counting unit of its classes. The count query compiled from it returns the exported origin and /24-pair counts in every window of two production days, and the JA4 class sizes wherever the WAF blocked no client. A new equality signal reaches the query as four triples, with no change to any code. The scope and its evidence chain (Listing 2) export as STIX 2.1 bundles that pass the OASIS validator in strict mode and cross the reference TAXII 2.1 server unchanged, save the extension definition it fails to serve. MISP’s importer drops the fingerprints, which STIX 2.1 carries only in an extension, and DOTS filters match only network and transport fields, so both would widen the scope to the endpoint or to address prefixes: fingerprint-scoped mitigation needs a standard property for JA4. Whether the derivation, which makes the verdict inspectable, shortens an analyst’s decision is untested.

VII. CONCLUSION AND FUTURE WORK

We treated the scope of an application-layer DDoS mitigation as a calibrated test over a session-centric knowledge graph. Against generated stealthy campaigns, per-session detection sits at chance, and cross-session features reach ROC AUC 0.93–0.98 only within one botnet structure. Scoping by the cluster’s most common property selects a legitimate fingerprint once the botnet spans several TLS stacks. A binomial enrichment test over normal traffic blocks 90% up to 25 stacks with no observed collateral and no label, and scopes the stacks real clients share, which a filter of unseen fingerprints cannot. On a CDN’s traffic, legitimate fleets set a floor on what the calibrated test can name: 16% of the busiest endpoint’s window under the binomial and 3% under a beta-binomial that models them, at the operating point of a calibrated z -score. A fresh day is consistent with the binomial configuration’s false-alarm rate, and in the stealthy regime the scope names the botnet that the volume trigger mostly misses. Triggers driven by the scope, HTTP/2 attacks and a captured stealthy campaign come next.

ACKNOWLEDGMENT

We thank Azion Technologies (<https://www.azion.com/>) for authorizing the fingerprint measurement of Section IV and the aggregated traffic statistics of Appendix E, exported without addresses, ports or URIs.

REFERENCES

- [1] National Institute of Standards and Technology, “CVE-2023-44487 detail,” National Vulnerability Database, 2023, accessed Sep. 25, 2026. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2023-44487>
- [2] J. Snellman and D. Iamartino, “How it works: The novel HTTP/2 ‘Rapid Reset’ DDoS attack,” Google Cloud Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack>
- [3] L. Pardue, “HTTP/2 rapid reset: Deconstructing the record-breaking attack,” Cloudflare Blog, Oct. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/technical-breakdown-http2-rapid-reset-ddos-attack/>
- [4] O. Yoachimik, “Hyper-volumetric DDoS attacks skyrocket: Cloudflare’s 2025 Q2 DDoS threat report,” Cloudflare Blog, Jul. 2025, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ddos-threat-report-for-2025-q2/>
- [5] E. Cambiaso, G. Papaleo, G. Chiola, and M. Aiello, “Slow DoS attacks: Definition and categorisation,” *International Journal of Trust Management in Computing and Communications*, vol. 1, no. 3–4, pp. 300–319, 2013.
- [6] C. Kemp, C. Calvert, T. M. Khoshgoftaar, and J. L. Leevy, “An approach to application-layer DoS detection,” *Journal of Big Data*, vol. 10, no. 1, p. 22, 2023.
- [7] M. Odusami, S. Misra, O. Abayomi-Alli, A. Abayomi-Alli, and L. Fernández-Sanz, “A survey and meta-analysis of application-layer distributed denial-of-service attack,” *International Journal of Communication Systems*, vol. 33, no. 18, p. e4603, 2020.
- [8] L. Belcastro, C. Carlucci, C. Cosentino, P. Liò, and F. Marozzo, “Enhancing network security using knowledge graphs and large language models for explainable threat detection,” *Future Generation Computer Systems*, vol. 176, p. 108160, 2026.
- [9] R. Bharathi and R. Sukanesh, “A PCA based framework for detection of application layer DDoS attacks,” *WSEAS Transactions on Information Science and Applications*, vol. 9, no. 12, pp. 389–398, 2012.
- [10] A. Martinetti, “Advancing threat intelligence: JA4 fingerprints and inter-request signals,” Cloudflare Blog, Aug. 2024, accessed Sep. 25, 2026. [Online]. Available: <https://blog.cloudflare.com/ja4-signals/>
- [11] Y. Jia, Y. Qi, H. Shang, R. Jiang, and A. Li, “A practical approach to constructing a knowledge graph for cybersecurity,” *Engineering*, vol. 4, no. 1, pp. 53–60, 2018.
- [12] V. Bonagiri, P. Priyadharshini, P. PanneerSelvi, S. Sahitya, R. Swamy, and P. Varalakshmi, “Practical applications of cybersecurity: A research paper on knowledge graphs for real-world security solutions,” in *Proc. 4th Asian Conf. on Innovation in Technology (ASIANCON)*, 2024.
- [13] P. E. Kaloroumakis and M. J. Smith, “Toward a knowledge graph of cybersecurity countermeasures,” The MITRE Corporation, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://d3fend.mitre.org/resources/D3FEND.pdf>
- [14] K. Liu, F. Wang, Z. Ding, S. Liang, Z. Yu, and Y. Zhou, “Recent progress of using knowledge graph for cybersecurity,” *Electronics*, vol. 11, no. 15, p. 2287, 2022.
- [15] L. Sadlek, M. Husák, and P. Čeleda, “Identification of device dependencies using link prediction,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2024, pp. 1–10.
- [16] X. Wang and R. Stadler, “IT intrusion detection using statistical learning and tested measurements,” in *Proc. IEEE/IFIP Network Operations and Management Symp. (NOMS)*, 2024, pp. 1–7.
- [17] A. Abou Daya, M. A. Salahuddin, N. Limam, and R. Boutaba, “A graph-based machine learning approach for bot detection,” in *Proc. IFIP/IEEE Symp. on Integrated Network and Service Management (IM)*, 2019, pp. 144–152.
- [18] Q. Cao, X. Yang, J. Yu, and C. Palow, “Uncovering large groups of active malicious accounts in online social networks,” in *Proc. ACM SIGSAC Conf. on Computer and Communications Security (CCS)*, 2014, pp. 477–488.
- [19] N. Tripathi and N. Hubballi, “Application layer denial-of-service attacks and defense mechanisms: A survey,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–33, 2021.
- [20] G. Fernandes Jr., J. J. P. C. Rodrigues, and M. L. Proença Jr., “Autonomous profile-based anomaly detection system using principal component analysis and flow analysis,” *Applied Soft Computing*, vol. 34, pp. 513–525, 2015.
- [21] S. Kandula, D. Katabi, M. Jacob, and A. Berger, “Botz-4-Sale: Surviving organized DDoS attacks that mimic flash crowds,” in *Proc. USENIX Symp. on Networked Systems Design and Implementation (NSDI)*, 2005.
- [22] M. Walfish, M. Vutukuru, H. Balakrishnan, D. Karger, and S. Shenker, “DDoS defense by offense,” in *Proc. ACM SIGCOMM*, 2006, pp. 303–314.
- [23] F. Silveira and C. Diot, “URCA: Pulling out anomalies by their root causes,” in *Proc. IEEE INFOCOM*, 2010.
- [24] Z. Li, M. Sanghi, Y. Chen, M.-Y. Kao, and B. Chavez, “Hamsa: Fast signature generation for zero-day polymorphic worms with provable attack resilience,” in *Proc. IEEE Symp. on Security and Privacy (S&P)*, 2006.
- [25] A. Mortensen, T. Reddy.K, F. Andreassen, N. Teague, and R. Compton, “DDoS Open Threat Signaling (DOTS) Architecture,” Internet Engineering Task Force, RFC 8811, Aug. 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8811>
- [26] M. Boucadair, J. Shallow, and T. Reddy.K, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Signal Channel Specification,” Internet Engineering Task Force, RFC 9132, Sep. 2021. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9132>
- [27] M. Boucadair and T. Reddy.K, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Data Channel Specification,” Internet Engineering Task Force, RFC 8783, May 2020. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8783>
- [28] M. Boucadair, T. Reddy.K, E. Doron, M. Chen, and J. Shallow, “Distributed Denial-of-Service Open Threat Signaling (DOTS) Telemetry,” Internet Engineering Task Force, RFC 9244, Sep. 2022. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9244>
- [29] W3C OWL Working Group, “OWL 2 web ontology language profiles, 2nd ed.” W3C Recommendation, 2012, accessed Sep. 25, 2026. [Online]. Available: <https://www.w3.org/TR/owl2-profiles/>
- [30] J. Althouse, “JA4+ network fingerprinting,” FoxIO Blog, Sep. 2023, accessed Sep. 25, 2026. [Online]. Available: <https://blog.foxio.io/ja4+-network-fingerprinting>
- [31] OASIS, “STIX version 2.1,” OASIS Standard, 2021, accessed Sep. 25, 2026. [Online]. Available: <https://docs.oasis-open.org/cti/stix/v2.1/stix-v2.1.html>
- [32] M. Antonakakis, T. April, M. Bailey, M. Bernhard, E. Bursztein, J. Cochran, Z. Durumeric, J. A. Halderman, L. Invernizzi, M. Kallitsis, D. Kumar, C. Lever, Z. Ma, J. Mason, D. Menscher, C. Seaman, N. Sullivan, K. Thomas, and Y. Zhou, “Understanding the Mirai botnet,” in *Proc. 26th USENIX Security Symp.*, 2017, pp. 1093–1110.
- [33] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward generating a new intrusion detection dataset and intrusion traffic characterization,” in *Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.
- [34] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, “CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,” *Sensors*, vol. 23, no. 13, p. 5941, 2023.
- [35] J. Jung, B. Krishnamurthy, and M. Rabinovich, “Flash crowds and denial of service attacks: Characterization and implications for CDNs and web sites,” in *Proc. 11th Int. World Wide Web Conf. (WWW)*, 2002, pp. 293–304.
- [36] E. Rescorla, K. Oku, N. Sullivan, and C. A. Wood, “TLS Encrypted Client Hello,” Internet Engineering Task Force, RFC 9849, Mar. 2026. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9849>
- [37] M. Shafi, A. H. Lashkari, V. Rodriguez, and R. Nevo, “Toward generating a new cloud-based distributed denial of service (DDoS) dataset and cloud intrusion traffic characterization,” *Information*, vol. 15, no. 4, p. 195, 2024.

APPENDIX A
RUNTIME CONSTRUCTION AND SUB-RELATION
INSTANTIATION

Algorithm 1 Runtime knowledge-graph construction

Require: request stream $\mathcal{R}$, ontology $\mathcal{O}$, window W

Ensure: knowledge graph KG persistent over W

```

1:  $KG \leftarrow \emptyset$ 
2: for each request  $r \in \mathcal{R}$  do
3:    $s, i, e \leftarrow \text{RESOLVESESSION/IDENTITY/ENDPOINT}(r)$ 
4:    $KG.add(s, i, e); KG.link(s, \text{hasIdentity}, i)$ 
5:    $KG.link(s, \text{targets}, e)$ 
6:   for each active  $s' \in S_W, s' \neq s$  do
7:     for each sub-relation  $i \in \mathcal{R}_{\text{rel}}$  do
8:       if  $\text{HOLDS}_i(s, s')$  then
9:          $KG.link(s, \text{relatedBy}_i, s')$ 
10:      end if
11:    end for
12:  end for
13:   $KG.purgeOutside(W)$ 
14: end for
15: return  $KG$ 

```

Algorithm 1 constructs the graph over the sliding window. It enumerates pairs for clarity: the equality-based relations are kept as class counts (Section III-F), and for the others the following procedures decide, for each candidate pair (s_a, s_b) , whether the edge is instantiated.

TLSFingerprint: exact equality of the full JA4 or, failing that, a near variant (same transport, TLS version and ALPN prefix, at most one symbol of difference in the ordered cipher or extension blocks), annotated $ja4_distance = 1$, which absorbs version drift within one client library without matching distinct stacks, $O(1)$ per pair. **ReusedIdentity**, **TemporalPattern** and **PayloadSignature**, specified but not exercised, match a shared cookie, token or username (JWTs by their sub claim), inter-arrival sequences within a length-normalized DTW distance τ_{DTW} (FastDTW, linear), and payload summaries above a cosine similarity τ_{payload} . **EndpointConvergence:** the same endpoint under path-pattern normalization ($/\text{api}/\text{users}/12345 \rightarrow / \text{api}/\text{users}/\{\text{id}\}$), annotated $exact = true$ on an exact-path match, $O(1)$ with a prefix tree. **NetworkProximity:** a shared IPv4 /24 (IPv6 /48), a shared ASN from current BGP tables, or both ($strong = true$), $O(1)$ with an inverted prefix index.

APPENDIX B
WEIGHT CALIBRATION

Calibration depends on the realism of the traffic: on laboratory captures, with a few dozen distinct JA4, optimization inverts the sign of the TLS signal, because benign sessions share fingerprints more than partially coordinated campaigns do. In the realistic scenario, the pool of 2000 benign fingerprints takes the Zipf shape of the 495 measured at one point of presence and exceeds the 76–818 each production endpoint shows. There, maximizing per-session attack-vs-benign ROC AUC of

```

# SWRL (Horn): instantiate one sub-relation
ApplicationSession(?a) ^ ApplicationSession(?b) ^
differentFrom(?a, ?b) ^
tlsJa4(?a, ?j) ^ tlsJa4(?b, ?j)
-> relatedByTLSFingerprint(?a, ?b)

```

```

# SPARQL: weighted aggregation from class sizes,
# counted in distinct origins
SELECT ?e (SUM(?w * ?n * (?n - 1) / 2) AS ?omega)
WHERE {
  { SELECT ?e ?c (COUNT(DISTINCT ?o) AS ?n)
    WHERE { ?s kg:targets ?e ; kg:inClass ?c ;
             kg:originatesFrom ?o . }
    GROUP BY ?e ?c }
  ?c kg:ofRelation ?rel .
  ?rel rdfs:subPropertyOf kg:relatedTo ;
  kg:coordinationWeight ?w .
  ?e kg:tauCluster ?tau . }
GROUP BY ?e ?tau
HAVING (SUM(?w * ?n * (?n - 1) / 2) >= ?tau)

```

Listing 1. Verdict as derivation: one Horn rule and the SPARQL aggregation.

$w_{\text{tls}} z(\text{ja4}) + w_{\text{ep}} z(\text{endpoint conv.}) + w_{\text{net}} z(\text{net prox.})$ over the grid $\{0.3 \dots 1.0\}^3$ (125 combinations, 60 scenarios, 91 500 sessions) gives $(w_{\text{tls}}=1.0, w_{\text{ep}}=0.3, w_{\text{net}}=0.3)$. Only the TLS fingerprint discriminates on its own (isolated AUC = 0.93, against 0.50 for endpoint convergence and 0.58 for network proximity), and the weights of Fig. 1 (1.0, 0.6, 0.3) attain the same optimal AUC = 0.943 and are insensitive to $\pm 20\%$ perturbation. The calibration thus corroborates the ordering, while the medium and low weights are not separately identifiable and the per-window rule detects as well with uniform weights (Appendix E). The popularity measurement is weighted per request, which if anything overstates concentration. Its head matches $\alpha = 1.5$ and its top ten $\alpha = 2.0$.

APPENDIX C
ADDITIONAL LIMITATIONS

Session and identity instrumentation. The approach assumes sessions, identities and TLS fingerprints can be extracted from traffic events, and partial or noisy instrumentation degrades it. Counting origins by source address undercounts users behind a carrier-grade NAT and overcounts clients that rotate IPv6 privacy addresses, whose natural unit is the /64.

Graph maintenance cost. The symbolic figures of Appendix D are an upper bound, and the production run of Appendix E is offline, over aggregated counts per endpoint, so a live CDN deployment still needs the partitioning of Appendix D, which we did not implement.

Coverage. HTTP/2 attacks and other application protocols (DNS, gRPC) need sub-relations beyond the Slow HTTP DoS family over HTTP/1.1, and $\Omega(S)$ instantiates only the three sub-relations the generator and captures expose. Identity reuse, temporal pattern and payload signature are specified (Appendix A) but not exercised, so their weights are untested.

Dataset breadth. Laboratory validation covers one dataset and one attack against KLAG, plus six attacks across CICIDS2017 and CIC-IoT2023. We found no independent capture of the stealthy regime: BCCC-cPacket-Cloud-DDoS-2024 names weak application-layer representation a deficiency of the datasets it surveys [37], yet its 17 DDoS scenarios

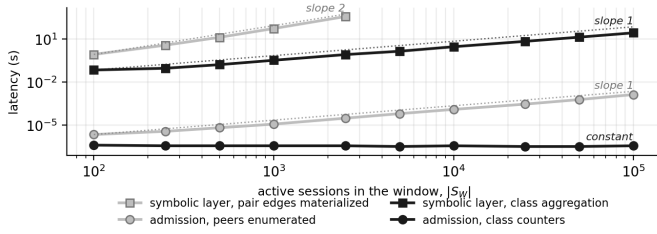


Fig. 3. Cost of both layers, with pair edges and with class counts.

are TCP-based. RT-IoT2022, CIC-DDoS2019 and a captured stealthy campaign remain future work.

Laboratory captures and KLAGe. On CIC-IoT2023 DDOS Slowloris, whose attack clusters hold seven origins in one of 143 /24 prefixes, a *lean* per-session baseline (three attributes) collapses to $F_1 = 0.179$ (AUC 0.551). A *strong* one (eight flow attributes) reaches 0.900 (AUC 0.987) and the cross-session representation 0.911 (AUC 0.982), because real Slowloris, unlike our stealthy campaigns, leaves a per-session flow signature. The comparison with KLAGe is favorable but *uncontrolled*: our strong baseline also beats the published 0.841, so the margin is not the representation’s, and the released code, which starts from a graph of unpublished construction and ships no weights, admits no controlled rerun.

Learned representation. Configuration (d) is trained and tested on a 70/30 split of one campaign’s sessions. Trained on 25 stacks and tested on 100 it reaches an AUC of 0.75, against 0.95–0.995 trained on the same M on other seeds. The generator draws every benign session before anything M -dependent, so cross- M tests use disjoint seeds: a same-seed split would score (a) at 0.88–0.94 from memorized copies.

Explanation validation. The evidence chains of the seven laboratory clusters pass completeness (every correlated session and activated sub-relation enumerated) and actionability (a scope coherent with the cluster discriminator) checks (7/7) when counted in sessions. Counted in origins, the five of CICIDS2017 hold one to three sources, below $k_{\min}$, and the two of CIC-IoT2023 hold seven. No user study tested them.

APPENDIX D

COST MODEL AND WINDOW SENSITIVITY

Admission happens per request and the Ω aggregation once per window, so we time the two layers separately, on a synthetic window in the realistic mix: 30% of sessions in one campaign sharing a botnet JA4, benign JA4 from a pool of 2000, eight endpoints, dispersed /24s. Each runs with pair edges, as the rules are written, and with class counts for the equality-based sub-relations, with medians over three repeats on one core. Both give the same $\Omega(S)$ for every endpoint, to within 7×10^{-12} . The symbolic layer runs on `rdflib` in place of Jena/TDB2, so its figures are an upper bound, and counting origins adds one hash-set lookup per admission.

With pair enumeration, median admission latency grows from $2.3 \mu\text{s}$ at $|S_W| = 100$ to $127 \mu\text{s}$ at $|S_W| = 10000$, tracking the candidate-pair count (148 to 6495 edges per

```
"@type": "kg:CoordinatedHTTPFlood",
"kg:clusterSize": 1991, "kg:coordinationScore": 1303422.8,
"kg:activatedSubRelations": [
{"@type": "kg:relatedByTLSPFingerprint",
"kg:coordinationWeight": 1.0, "kg:linkedPairs": 114710,
"kg:weightedContribution": 114710.0},
{"@type": "kg:relatedByEndpointConvergence",
"kg:coordinationWeight": 0.6, "kg:linkedPairs": 1981045,
"kg:weightedContribution": 1188627.0},
{"@type": "kg:relatedByNetworkProximity",
"kg:coordinationWeight": 0.3, "kg:linkedPairs": 286,
"kg:weightedContribution": 85.8}],
"kg:derivedMitigationScope": {
"kg:endpoint": "10.0.0.1:443",
"kg:tlsJa4": ["t13d1516h2_synth_0001_00", ...]}
```

Listing 2. Evidence chain of a canonical campaign ($M = 25$, $K = 1000$), counted in origins. The scope lists 25 fingerprints, abridged.

admission) at 15–21 ns per pair. With class counters it stays between 0.33 and $0.42 \mu\text{s}$ from 100 to 100 000 sessions (Fig. 3, dotted lines are reference slopes). Materializing pair edges costs 0.84 s at $|S_W| = 100$ and 52.8 s at 1000, within 198–309 μs per RDF edge, linear in edges and quadratic in sessions, whereas class aggregation, one membership triple per session and sub-relation and the query of Listing 1, takes 0.33 s at 1000 sessions, 161 times faster, and 26.4 s at 100 000. The quadratic term thus lives in the value of $\Omega(S)$, which counts pairs, and in the pair edges of the non-transitive sub-relations, whose candidates the indexes restrict. The evidence chain (Listing 2) needs only the pair counts the classes give.

On the generator, W is the gap that splits an endpoint’s sessions into detection clusters. Over $W = 60$ –1800 s on the same scenarios ($K = 1000$, $n = 10$) the AUC of (d) stays at 0.976–0.978 and that of (c) at 0.663–0.664. The campaign’s cluster holds 1977–1998 sessions at every W , while the mean cluster grows from 132 to 867, and the discriminative feature is the *fraction* of that cluster sharing one JA4. The insensitivity is thus a property of the generator.

Partitioning at CDN Scale

Condition (ii), one endpoint per S , makes the endpoint the natural shard key: partitioning by it evaluates the rule *exactly*, with no cross-shard traffic. A campaign spreading one JA4 across many endpoints then accumulates too few pairs in any shard, and no single key preserves all six sub-relations. A two-tier design, unimplemented here, would add a global sketch over high-weight identifiers, and its recall must be measured before a CDN-scale deployment.

APPENDIX E

THE RULE PER WINDOW

Here the rule runs as deployed, per window of $W = 300$ s and endpoint, and no threshold sees an attack or a label. τ_{cluster} is the 99th percentile of Ω over the windows of 30 attack-free runs, which also form the profile. False alarms are counted on 30 more, detection on the canonical campaigns ($\alpha = 1.5$, $M = 25$) over windows with at least $k_{\min} = 5$ attackers, and a flash crowd retimes 25, 50 or 100 legitimate sessions of a third run into one window. We spread benign starts over 60 min, since the generator packs them into the first minutes.

TABLE V
THE RULE PER FIVE-MINUTE WINDOW, 30 RUNS PER ROW.

Windows	n_W	$\Omega \geq \tau$	Ω and enrichment		
			Fires	Cov.	Coll.
Attack, $K = 1000$	90	77.8%	77.8%	79.8%	0.0%
Attack, $K = 50$	35	85.7%	85.7%	75.3%	0.0%
Clean	360	0.6%	0.0%	—	—
Flash crowd, 25	30	80.0%	0.0%	—	—
Flash crowd, 50	30	100%	0.0%	—	—
Flash crowd, 100	30	100%	0.0%	—	—

Cov.: median share of the window’s attackers the scope blocks. Coll.: largest share of its legitimate sessions blocked.

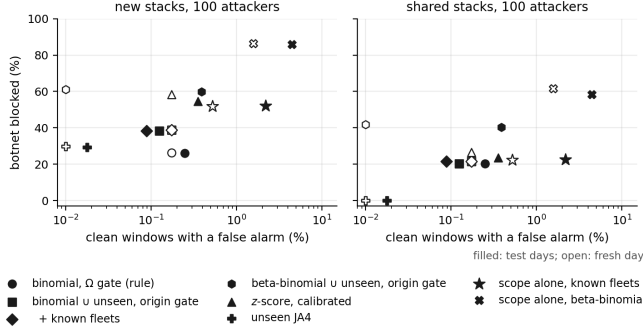


Fig. 4. Operating points on production traffic, 100 attackers per window (a zero rate is drawn at 0.01%).

A flash crowd adds mass the way a campaign does. The test tells them apart because these crowds draw users from the benign mix the profile describes, while a surge on one client type is enriched, as production fleets show. With the test the rule fires on no clean window (95% upper bound 0.8%) and no flash crowd, and blocks a median 75–80% of the attackers in every attack window Ω flags, hitting no legitimate session. With uniform weights Ω catches at least as many attack windows (78.9% and 85.7%), and without its endpoint term 33.3% and none, while still flagging 53–100% of flash crowds. A distinct-origin gate at its 99th percentile catches 82.2% and 85.7% and, with the test, flags no clean window or flash crowd.

Production traffic. The exports come from the four endpoints of the operator’s own services whose windows hold at least $k_{\min}$ origins. Clean traffic excludes clients the WAF blocked, and the exports carry no prefixes, so the scope is judged on JA4 alone. The injected botnet puts 90% of its attackers on 25 stacks, with /24s from 2000 prefixes disjoint from real ones. Fleets, such as a probe fleet active only in business hours, push λ_e to the levels of Table IV, and the scope alone then names a filter in 1.5–6.3% of the next day’s windows. On a clean window of E2 or E3 the rule then names a fleet, blocking about half of that window’s clients. The origin gate halves these false alarms: the eight clean windows that only Ω admits, with the scope firing, carry 33% of their Ω outside the endpoint term, against 12% elsewhere, since a fleet concentrates on one fingerprint and lifts the TLS term.

WAF agreement. Of the clients the scope would block on

the full traffic, the WAF also blocked 84% and 96% on the two endpoints where the scope names no fleet and 1.5% on the one where it does. The scope matches 16.7% of all the clients the WAF blocked.

Other baselines. Uncalibrated, the z -score fires on 2.6% of clean windows and 64.5% of flash crowds, and a z -score against each fingerprint’s own history on half of them.

Overdispersed background. The beta-binomial variant gives each fingerprint of the profile an intra-window correlation ϕ , a moment estimate over the calibration windows from $\text{Var}[c] = nb(1-b)(1+(n-1)\phi)$, clipped to $[0, 0.99]$, and tests its count against a beta-binomial of the same mean. A fingerprint absent from the profile keeps the binomial, and the level is calibrated as before.

Of the four known-fleet shares tried on the first three test days, 5% was the only one that did not raise false alarms. On the last two it lifts enrichment on E1 at a tenth of the typical window from 0.7% to 6.3% of the attackers, at 7 false alarms against 8, and leaves E2 to E4 unchanged. Smaller shares block more but fire twice as often, on lighter filters (a median 11–12% of clients against 51%). The calibrated baselines take as threshold the 99th percentile of the largest z over the calibration windows, never below 3. A trigger on the scope alone fires, with known fleets, on 2.2% of pooled clean windows and 0.5% on the fresh day, against 3.5% and 8.8% without them (Fig. 4). An hourly profile did not help, and $\rho = 2$ or 5 moves no pooled firing rate by over 0.3 points and no blocked share by over 4.5, while a collateral median shifts by up to 7.2. Flash crowds of 1000 users trigger the rule in 21.8% of windows, the z -score in 86.8%.

APPENDIX F REPRODUCIBILITY

The reference implementation is available at <https://github.com/NatanaelOliveira/knowledge-graph-ddos-article> under a permissive license. `make compile-check` `stix-ingest` checks the compiled count query on generated sessions and passes the exported bundles through the OASIS STIX 2.1 validator in strict mode, TAXII 2.1 and MISP’s importer. The fresh day’s protocol was written down, with the hashes of its exports, before the day was read (`fresh_day_protocol.md`), and `make production-tables` recomputes every production number. Both scope derivations ship side by side, so the negative result of Section V-C is reproducible rather than asserted, and labels enter only the evaluation: profiles and levels come from attack-free runs, and the scripts behind Table II pick the largest- Ω cluster with no label. Seeds are fixed and Makefiles drive every stage, so each table and data figure regenerates from one command, except the inputs of Tables III and IV, the operator’s aggregates, whose anonymized summary ships with the artifact and is checked against the tables. The OWL ontology ships in RDF/XML with its `coordinationWeight` annotations, and we intend to submit the artifacts to the Artifact Evaluation track.